Term report for the implementation of Multi-agent reinforcement learning for redundant robot control in task-space

Kothawale Chetan Dattatraya
M.Tech. Robotics and Mobility Systems
Indian Institute of Technology, Jodhpur
Rajasthan, India
m24irm003@iitj.ac.in

Vishwanath R
M.Tech. Robotics and Mobility Systems
Indian Institute of Technology, Jodhpur
Rajasthan, India
m24irm010@iitj.ac.in

***Abstract*—Task-space control in redundant robotic manipulators requires solving the inverse kinematics (IK) problem, often through analytical methods involving the Jacobian matrix. However, due to the presence of excess joint degrees-of-freedom compared to Cartesian dimensions, obtaining stable and unique solutions becomes challenging, particularly near singularities. In this work, we explore and compare three different approaches to address this issue: the Moore-Penrose pseudo-inverse method, a Jacobian-based singularity avoidance strategy, and a fully cooperative Multi-Agent Reinforcement Learning (MARL) framework. In the MARL approach, each joint is treated as an independent agent, and a critic-only Q-learning mechanism is employed to learn coordinated joint behaviours without relying on large datasets or function approximators. Our comparative analysis highlights the advantages of MARL in handling redundancy and achieving stable convergence, while also demonstrating the effectiveness of analytical methods under specific conditions. Experimental results validate the potential of MARL as a viable alternative to traditional IK solutions.**

I. Introduction

This paper *Multi-agent reinforcement learning for redundant robot control in task-space*[1] puts forth the challenges faced in the task-space control of a robot by considering an example of a 4 degrees of freedom manipulator arm. In task-space control, traditional methods usually involve inverse kinematics solutions or the Jacobian matrix that facilitates the robot to reach the desired pose in the workspace. In practice, redundant robots have fewer degrees of freedom than joint angles, the control inputs that bring multiple solutions to solving equations. This problem stems from the fact that Jacobian matrices are square for non-redundant robots and yield unique solutions. However, the original aim of having extra degrees of freedom is to facilitate complex control and achieve complex tasks. Recent advancements in machine learning have introduced new approaches to addressing these challenges. Neural networks can learn solutions for inverse kinematics but require extensive training data and offline computation. On the other hand, reinforcement learning (RL) shows promise for model-free adaptive control; however, standard single-agent RL methods encounter the curse of dimensionality when applied to multi-joint systems. Multi-agent reinforcement learning (MARL) provides a framework to address this by treating each joint as an independent learning agent. This paper presents a comprehensive comparison of Jacobian-based control methods, specifically the Moore-Penrose pseudo-inverse and singularity-avoidance techniques, with Q-learning-based multi-agent reinforcement learning (MARL) for controlling redundant manipulators.

II. Methodology

A. Robot setup

The authors of this paper used a real-life exoskeleton robot that comprises four joints, one end-effector, and four links. This has been represented very well by the authors and is also depicted in Fig. 1. With Fig. 1 as a reference, Denavit-Hartenbug parameters can be derived as described in Table 1. The same values can also be verified with the experimental results presented in the paper[1].

TABLE I
Denavit-Hartenberg parameters for the 4 DoF robot

$Joint_i$	θ_i	d_i	a_i	α_i
1	q_1	l_1	0	$\frac{\pi}{2}$
2	q_2	0	0	$\frac{\pi}{2}$
3	q_3	0	l_3	$\frac{-\pi}{2}$
4	q_4	0	l_4	0

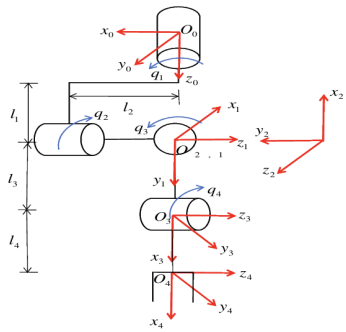

Fig. 1. D-H frame for the robot as presented in paper[1]

B. Traditional methods

The Jacobian is a multidimensional form of the derivative[2]. For a function f of three independent variables x_1, x_2, x_3, it can be represented in vector format as

$$Y = F(X)$$

where X is a column matrix consisting of x_1, x_2, x_3 and Y is also a column matrix consisting of respective function values of X. If the differentials of Y have to be calculated, then the same can be represented by differentiating the vector notation, which can be expressed as

$$\delta Y = \frac{\delta F}{\delta X}.\delta X$$

This term $\frac{\delta F}{\delta X}$ is known as Jacobian, J. The Jacobian matrix will be square if the number of elements in the column vector X and Y are the same. The column matrix X and Y represent the joint-space and task-space variables, respectively. For this configuration of the robot, there are three task-space variables (x, y, z) and four joint-space variables (q_1, q_2, q_3, q_4). Since the number of task-space variables is lesser than joint-space variables, the formed Jacobian matrix will not be square. Hence, the matrix loses its rank at singular points.

The authors of the paper[1] have implemented PID control along with the Jacobian-based augmented method (J^+AJ), Jacobian-based singularity avoidance (J^+SA) and Jacobian-based singular value decomposition (J^+SVD) keeping these as the base references which then helped them understand the results from the multi-layer neural network and reinforcement learning methods. This report aims to implement the Pseudo-Inverse Jacobian method, which is derived using an approximation of $\delta = 0.000001$, and we consider that to be the foundation for comparison. We have also implemented Jacobian-based singularity avoidance (J^+SA), where the Jacobian is again derived using the same approximation method. The authors have not explicitly mentioned their implementations of these Jacobian methods, so we assume they would have implemented them in a simulation environment.

We define the positional error $\mathbf{e}$ as the current position x and desired position x_d. Hence, PID law for this control can be formulated as

$$\mathbf{u} = K_p\mathbf{e} + K_i \int \mathbf{e}\, dt + K_d \frac{d\mathbf{e}}{dt}$$

where $\mathbf{e} = \mathbf{x}_d - \mathbf{x}$ is the position error. The integral term is updated as:

$$\mathbf{e}_{\text{int}} = \mathbf{e}_{\text{int}} + \mathbf{e} \cdot dt$$

and the derivative term is approximated as:

$$\frac{d\mathbf{e}}{dt} = \frac{\mathbf{e} - \mathbf{e}_{\text{prev}}}{dt}$$

As discussed earlier, since the Jacobian matrix is not square, we have to calculate its pseudo-inverse using Moore-Penrose pseudo-inverse, which can be expressed as:

$$\dot{\mathbf{q}} = J^{\dagger}\mathbf{u}, \quad J^{\dagger} = J^T(JJ^T)^{-1}$$

Although we find pseudo-inverse using the Moore-Penrose pseudo-inverse method, the Jacobian matrix still loses its rank near singularity points, causing instability. In singularities, JJ^T becomes unstable, causing infinite velocities. This is eliminated by introducing a damping factor multiplied by an identity matrix and added to the Jacobian matrix, which will compensate for the singularity points. This method is called as Jacobian-based singularity avoidance (J^+SA) using Damped Least Squares (DLS) .

$$J_{\lambda}^{+} = J^T(JJ^T + \lambda^2 I)^{-1}$$

where λ is a small positive constant. Hence, the modified joint velocity equation is:

$$\dot{q} = J_{\lambda}^{+}\ \dot{x}$$

Computing the Jacobian matrix ensures accurate trajectory tracking, but computing this analytical Jacobian method is difficult. Hence, we use numerical techniques that can approximate the Jacobian. We assume the perturbation δ to be infinitesimally small such that it is as good as computing for every single point in the trajectory. Given a small perturbation δ in each joint variable, the Jacobian columns are estimated as:

$$J(:, i) \approx \frac{f(\mathbf{q} + \delta\mathbf{e}_i) - f(\mathbf{q})}{\delta}$$

where $\mathbf{e}_i$ is the unit vector in the i-th joint direction. This approach avoids symbolic differentiation and allows for real-time Jacobian estimation.

C. Reinforcement Learning based method

To address the problems with Jacobian-based controllers, particularly when dealing with singularities and inaccuracies from model assumptions, we have also chosen Reinforcement

Learning(RL) based methods like the authors of the paper[1]. They have proposed a novel actor-critic based Multi-agent Reinforcement Learning(MARL) method. We have chosen to explore along the same lines but try something different from what they have proposed. We have chosen to experiment with a Q-learning based approach that uses only a critic.
In the multi-agent formulation, each joint of the 4-degrees-of-freedom (4-DoF) manipulator is treated as an independent agent operating within a shared environment. Each agent's objective is to take actions that help the manipulator's end-effector reach a target position in task space. The goal here is to learn a policy that minimizes the error between the current end-effector position x and the desired position x_d. This approach, which combines decoupling with coordination, enables distributed learning and policy optimisation.

We implement a classical Q-learning update mechanism with Temporal Difference (TD) Learning and shared critic:

$$Q(s,a) \leftarrow Q(s,a) + \alpha_a \cdot TDerror \tag{1}$$

$$TDerror = r + \gamma \cdot V(s') - V(s) \tag{2}$$

Critic Update (Shared V-Value):

$$V(s) \leftarrow V(s) + \alpha_c \cdot TDerror \tag{3}$$

where, α_a, α_c are learning rates for actor and critic respectively, γ is the discount factor and r is the reward received for the transition (s, a, s'). The policy is updated online based on experience gathered during simulated episodes.
The reward function includes both Position Error Penalty (to encourage reaching the goal) and a Movement Penalty (to discourage abrupt joint motions).

$$r = -\|e_{\text{new}}\|^2 - 0.1 \cdot \|\Delta q\|^2 \tag{4}$$

where, $e_{\text{new}} = x_d - x$ is the position error, Δq is the change in joint configuration. This encourages the agent to converge to the target position smoothly.

III. Simulation and Observation

The proposed PID control method was implemented in the MATLAB simulation environment, using the *ode45* solver for numerical integration. The simulation was carried out for 3 seconds, tracking the desired position of the end effector $\mathbf{x}_d = [-0.24, -0.196, 0.207]^T$.

Figures 2 and 3 show the end-effector trajectory in 3D task space (x, y, z). The controller minimises the position error, as Figures 5 and 6 illustrate.

Figures 4 and 7 shows the performance of the MARL controller in reaching the desired position. The convergence occurs more quickly compared to the Jacobian-based methods, although some oscillations were observed due to untuned hyperparameters. In comparison to the Moore-Penrose and J^+SA methods, reinforcement learning (RL) demonstrated better performance regarding stability and settling time. However, it is important to note that RL requires training time and careful tuning of rewards and hyperparameters.

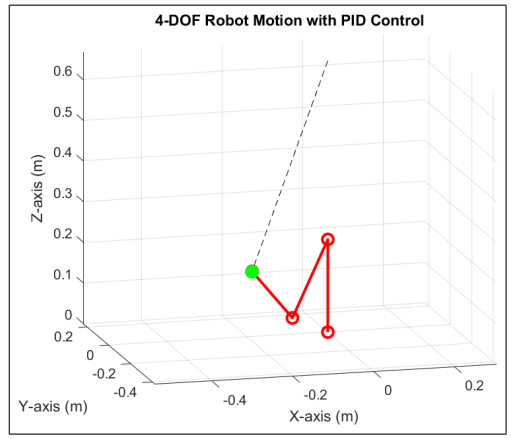

Fig. 2. End-effector trajectory tracking using Moore-Penrose pseudo-inverse Jacobian method

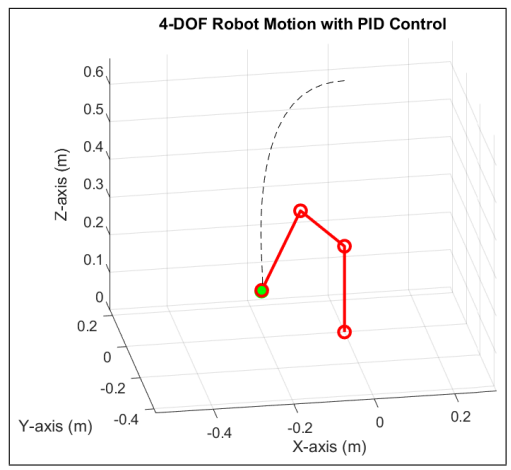

Fig. 3. End-effector trajectory tracking using Jacobian-based singularity avoidance method

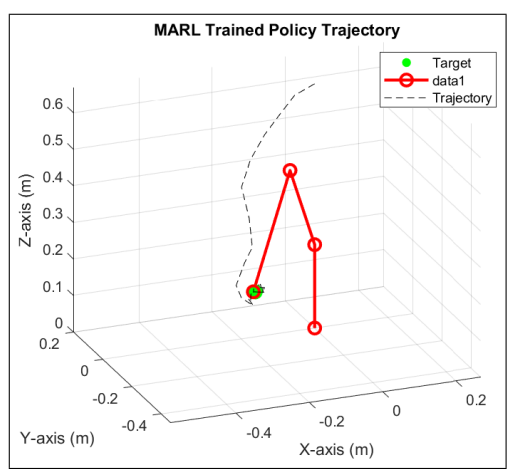

Fig. 4. End-effector trajectory tracking using RL based Q-Learning method

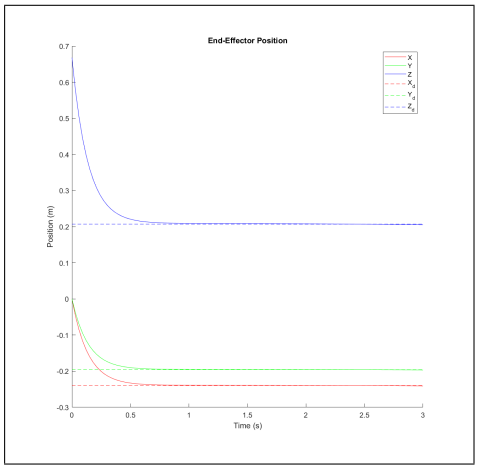

Fig. 5. Position error (Z) over time for Moore-Penrose pseudo-inverse Jacobian method

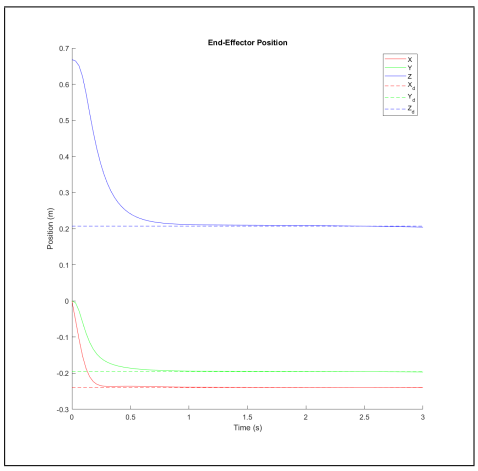

Fig. 6. Position error (Z) over time for Jacobian-based singularity avoidance method

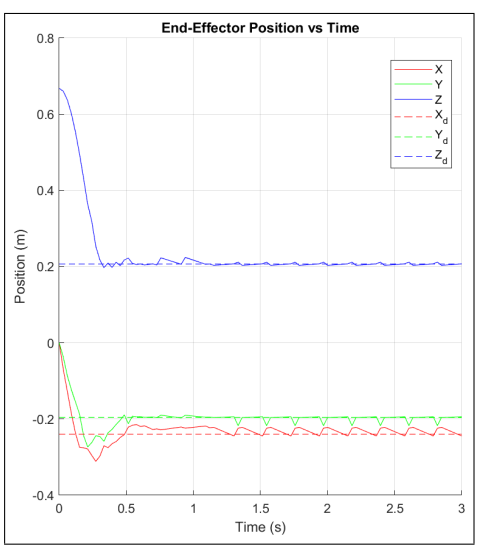

Fig. 7. Position error (Z) over time for RL based Q-learning method

IV. Discussion

During the presentation of our implementation, we received good amount of insights about how we could have approached the problem and the issues with our implementation of an algorithm which works with discrete actions on a continuous space.

V. Conclusion

This report presents both Jacobian-based and Reinforcement Learning-based control strategies for a 4-DoF manipulator. While Jacobian methods provide a fast and direct approach, they suffer near singularities and require explicit model knowledge. On the other hand, our implementation of Q-learning based MARL demonstrates potential in achieving better control performance with adaptability and robustness, albeit at the cost of training complexity.

VI. Individual Contributions

After thorough reading and discussion about the paper[1], the Moore-Penrose pseudo-inverse method was understood and implemented by *Vishwanath R*, the second author of this report. The first author, *Kothawale Chetan Dattatraya*, has mitigated the issue of unstability at singular points by implementing Jacobian-based singular avoidance ($J^{+}SA$). In addition, the first author also designed the environment and the agent for the MARL method, while the second author designed and tuned the reward function for optimal performance. All experiments were conducted collaboratively, with both authors contributing equally to the project's overall implementation and analysis.

References

[1] A. Perrusquía, W. Yu, and X. Li, "Multi-agent reinforcement learning for redundant robot control in task-space," *International Journal of Machine Learning and Cybernetics*, vol. 12, no. 1, pp. 231–241, 2020. [Online]. Available: https://doi.org/10.1007/s13042-020-01167-7

[2] J. Craig, *Introduction to Robotics: Mechanics and Control*. Pearson, 2017.